

CPU Scheduling Algorithms

1 First-Come, First-Served (FCFS)

The simplest non-preemptive CPU scheduling algorithm. Processes are allocated to the CPU in the exact order of their arrival. It is easy to implement but can suffer from the "convoy effect."

Input & Output Format

Input: Integer n. Next n lines: Arrival Time (AT) and Burst Time (BT).

Output: Table of PID, AT, BT, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define nl cout << endl;
4
5 struct Process{ int pid, at, bt, ct, tat, wt;
6     void show(){ cout << pid << "\t" << at << "\t" << bt << "\t" << ct << "\t" << tat << "\t" << wt << endl;
7         ; }
8 };
9
10 struct Gantt{ int pid, start, end; };
11
12 void showGanttChart(vector<Gantt> g){
13     cout << "\n\nGantt Chart\n\n";
14     for(auto x : g) cout << "| P" << x.pid << " ";
15     cout << "\n\n" << g[0].start;
16     for(auto x : g) cout << setw(5) << x.end;
17     nl;
18 }
19
20 int32_t main(){
21     int n = 0; cin >> n;
22     vector<Process> p(n);
23     for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt; }
24
25     sort(p.begin(), p.end(), [](Process a, Process b){
26         return a.at == b.at ? a.pid < b.pid : a.at < b.at;
27     });
28
29     int curTime = 0; vector<Gantt> g;
30     for(int i = 0; i < n; i++){
31         if(curTime < p[i].at) curTime = p[i].at;
32         curTime += p[i].bt;
33         g.push_back({p[i].pid, curTime - p[i].bt, curTime});
34         p[i].ct = curTime; p[i].tat = p[i].ct - p[i].at; p[i].wt = p[i].tat - p[i].bt;
35     }
36
37     cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
38     double totTat = 0, totWt = 0;
39     for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
40     cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
41     showGanttChart(g);
42 }
```

2 Shortest Job First (SJF) - Non-Preemptive

A non-preemptive algorithm that selects the waiting process with the smallest execution time (Burst Time). Resolves ties using arrival time.

Input & Output Format

Input: Integer n. Next n lines: Arrival Time (AT) and Burst Time (BT).

Output: Table of PID, AT, BT, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define nl cout << endl;
4
5 struct Process{ int pid, at, bt, ct, tat, wt; bool finished = false;
6     void show(){ cout << pid << "\t" << at << "\t" << bt << "\t" << ct << "\t" << tat << "\t" << wt << endl;
7         ; }
8 }
```

```

7 };
8 struct Gantt{ int pid, start, end; };
9
10 void showGanttChart(vector<Gantt> g){
11     cout << "\n\nGantt Chart\n\n";
12     for(auto x : g) cout <<"| P" << x.pid <<" ";
13     cout << "\n" << g[0].start;
14     for(auto x : g) cout << setw(5) << x.end;
15     nl;
16 }
17
18 int32_t main(){
19     int n = 0; cin >> n;
20     vector<Process> p(n); vector<Gantt> g;
21     for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt; }
22
23     int completed = 0, curTime = 0;
24     while(completed < n){
25         int selected = -1;
26         for(int i = 0; i < n; i++){
27             if(!p[i].finished and p[i].at <= curTime){
28                 if(selected == -1 or p[i].bt < p[selected].bt or (p[i].bt == p[selected].bt and p[i].at < p[
29                     selected].at)) selected = i;
30             }
31         }
32         if(selected == -1){ curTime++; continue; }
33
34         curTime += p[selected].bt;
35         g.push_back({p[selected].pid, curTime - p[selected].bt, curTime});
36         p[selected].ct = curTime; p[selected].tat = p[selected].ct - p[selected].at; p[selected].wt = p[
37             selected].tat - p[selected].bt;
38         p[selected].finished = true; completed++;
39     }
40
41     cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
42     double totTat = 0, totWt = 0;
43     for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
44     cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
45     showGanttChart(g);
46 }

```

3 Round Robin (RR)

A preemptive scheduling algorithm where each process is assigned a fixed time slot called a "Time Quantum." If it doesn't finish, it moves to the back of the queue.

Input & Output Format

Input: Integer n. Next n lines: AT & BT. Next line: Time Quantum (tq).

Output: Table of PID, AT, BT, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define nl cout << endl;
4
5 struct Process{ int pid, at, bt, ct, tat, wt;
6     void show(){ cout << pid << "\t" << at << "\t" << bt << "\t" << ct << "\t" << tat << "\t" << wt << endl;
7         ; }
8 };
9
10 struct Gantt{ int pid, start, end; };
11
12 void showGanttChart(vector<Gantt> g){
13     cout << "\n\nGantt Chart\n\n";
14     for(auto x : g) cout << "| P" << x.pid << " ";
15     cout << "\n\n" << g[0].start;
16     for(auto x : g) cout << setw(5) << x.end;
17     nl;
18 }
19
20 int32_t main(){
21     int n = 0; cin >> n;
22     vector<Gantt> g; vector<Process> p(n);
23     for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt; }
24
25     int tq = 0; cin >> tq;
26     vector<int> rem(n); for(int i = 0; i < n; i++) rem[i] = p[i].bt;
27
28     sort(p.begin(), p.end(), [](Process a, Process b){ return a.at == b.at ? a.pid < b.pid : a.at < b.at; });
29
30     int curTime = 0, completed = 0, i = 0; queue<int> q;
31     while(completed < n){
32         if(q.empty() and i < n and curTime < p[i].at) curTime = p[i].at;
33         while(i < n and p[i].at <= curTime){ q.push(i); i++; }
34
35         int idx = q.front(); q.pop();
36         int excTime = min(tq, rem[idx]);
37         rem[idx] -= excTime;
38         g.push_back({p[idx].pid, curTime, curTime + excTime});
39         curTime += excTime;
40
41         while(i < n and p[i].at <= curTime){ q.push(i); i++; }
42
43         if(rem[idx]){ q.push(idx); }
44         else{
45             completed++;
46             p[idx].ct = curTime; p[idx].tat = p[idx].ct - p[idx].at; p[idx].wt = p[idx].tat - p[idx].bt;
47         }
48     }
49
50     cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
51     double totTat = 0, totWt = 0;
52     for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
53     cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
54     showGanttChart(g);
55 }
```

4 Priority Scheduling (Non-Preemptive)

The CPU is allocated to the process with the highest priority (smaller integer indicates a higher priority). Once a process starts, it runs to completion.

Input & Output Format

Input: Integer n. Next n lines: AT, BT, & Priority.

Output: Table of PID, AT, BT, Priority, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define nl cout << endl;
4
5 struct Process{ int pid, at, bt, priority, ct, tat, wt; bool finished = false;
6     void show(){ cout << pid << "\t" << at << "\t" << bt << "\t" << priority << "\t\t\t" << ct << "\t" <<
7         tat << "\t" << wt << endl; }
8 };
9 struct Gantt{ int pid, start, end; };
10
11 void showGanttChart(vector<Gantt> g){
12     cout << "\n\nGantt Chart\n\n";
13     for(auto x : g) cout << "| P" << x.pid << " ";
14     cout << "\n" << g[0].start;
15     for(auto x : g) cout << setw(5) << x.end;
16     nl;
17 }
18
19 int32_t main(){
20     int n = 0; cin >> n;
21     vector<Process> p(n); vector<Gantt> g;
22     for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt >> p[i].priority; }
23
24     int curTime = 0, completed = 0;
25     while(completed < n){
26         int selected = -1;
27         for(int i = 0; i < n; i++){
28             if(!p[i].finished and p[i].at <= curTime){
29                 if(selected == -1 or p[i].priority < p[selected].priority) selected = i;
30             }
31         }
32         if(selected == -1){ curTime++; continue; }
33
34         g.push_back({p[selected].pid, curTime, curTime + p[selected].bt});
35         curTime += p[selected].bt;
36         p[selected].ct = curTime; p[selected].tat = p[selected].ct - p[selected].at; p[selected].wt = p[
37         selected].tat - p[selected].bt;
38         p[selected].finished = true; completed++;
39     }
40
41     cout << "\nPID\tAT\tBT\tPriority\tCT\tTAT\tWT\n";
42     double totTat = 0, totWt = 0;
43     for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
44     cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
45     showGanttChart(g);
46 }
```

5 Shortest Remaining Time First (SRTF)

The preemptive version of SJF. It schedules the process with the least remaining burst time. Preempts currently running process if a shorter job arrives.

Input & Output Format

Input: Integer n. Next n lines: AT & BT.

Output: Table of PID, AT, BT, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define nl cout << endl;
4
5 struct Process{ int pid, at, bt, ct, tat, wt;
6     void show(){ cout << pid << "\t" << at << "\t" << bt << "\t" << ct << "\t" << tat << "\t" << wt << endl;
7         ; }
8 };
9
10 struct Gantt{ int pid, start, end; };
11
12 void showGanttChart(vector<Gantt> g){
13     cout << "\n\nGantt Chart\n\n";
14     for(auto x : g) cout << "| P" << x.pid << " ";
15     cout << "\n\n" << g[0].start;
16     for(auto x : g) cout << setw(5) << x.end;
17     nl;
18 }
19
20 int32_t main(){
21     int n = 0; cin >> n;
22     vector<Gantt> g; vector<Process> p(n); vector<int> rem(n);
23     for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt; rem[i] = p[i].bt; }
24
25     int curTime = 0, completed = 0;
26     while(completed < n){
27         int selected = -1;
28         for(int i = 0; i < n; i++){
29             if(p[i].at <= curTime and rem[i]){
30                 if(selected == -1 or rem[i] < rem[selected] or (rem[i] == rem[selected] and p[i].at < p[
31                     selected].at)) selected = i;
32             }
33         }
34         if(selected == -1){ curTime++; continue; }
35         rem[selected]--;
36
37         if(!g.empty() && g.back().pid == p[selected].pid) g.back().end = curTime + 1;
38         else g.push_back({p[selected].pid, curTime, curTime + 1});
39
40         curTime++;
41         if(rem[selected] == 0){
42             p[selected].ct = curTime; p[selected].tat = p[selected].ct - p[selected].at; p[selected].wt =
43             p[selected].tat - p[selected].bt;
44             completed++;
45         }
46     }
47
48     cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
49     double totTat = 0, totWt = 0;
50     for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
51     cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
52     showGanttChart(g);
53 }
```

6 Priority Scheduling (Preemptive)

If a new process arrives in the ready queue with a higher priority (smaller number) than the currently executing process, the CPU is immediately given to the new process.

Input & Output Format

Input: Integer n. Next n lines: AT, BT, & Priority.

Output: Table of PID, AT, BT, Priority, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define nl cout << endl;
4
5 struct Process{ int pid, at, bt, priority, ct, tat, wt, rem_bt; bool finished = false;
6 void show(){ cout << pid << "\t" << at << "\t" << bt << "\t" << priority << "\t\t\t" << ct << "\t" <<
7 tat << "\t" << wt << endl; }
8 };
9
10 struct Gantt{ int pid, start, end; };
11
12 void showGanttChart(vector<Gantt> g){
13     cout << "\n\nGantt Chart\n\n";
14     for(auto x : g) cout << "| P" << x.pid << " ";
15     cout << "\n" << g[0].start;
16     for(auto x : g) cout << setw(5) << x.end;
17     nl;
18 }
19
20 int32_t main(){
21     int n = 0; cin >> n;
22     vector<Process> p(n); vector<Gantt> g;
23     for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt >> p[i].priority; p[i].
24         rem_bt = p[i].bt; }
25
26     int curTime = 0, completed = 0;
27     while(completed < n){
28         int selected = -1;
29         for(int i = 0; i < n; i++){
30             if(!p[i].finished and p[i].at <= curTime){
31                 if(selected == -1 or p[i].priority < p[selected].priority) selected = i;
32             }
33         }
34         if(selected == -1){ curTime++; continue; }
35
36         curTime++; p[selected].rem_bt--;
37
38         if(!g.empty() and g.back().pid == selected + 1) g.back().end = curTime;
39         else g.push_back({selected + 1, curTime-1, curTime});
40
41         if(p[selected].rem_bt == 0){
42             p[selected].ct = curTime; p[selected].tat = p[selected].ct - p[selected].at; p[selected].wt =
43             p[selected].tat - p[selected].bt;
44             p[selected].finished = true; completed++;
45         }
46     }
47
48     cout << "\nPID\tAT\tBT\tPriority\tCT\tTAT\tWT\n";
49     double totTat = 0, totWt = 0;
50     for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
51     cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
52     showGanttChart(g);
53 }

```

7 Multilevel Queue Scheduling

Ready queue is partitioned. Queue 1 processes run FCFS, and Queue 2 processes run Round Robin (Time Quantum = 2). Queue 1 has absolute priority over Queue 2.

Input & Output Format

Input: Integer n. Next n lines: AT, BT, & Queue Number (1 or 2).

Output: Table of PID, AT, BT, Queue, CT, TAT, WT; Average TAT & WT; Gant Chart (with Idle).

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define nl cout << endl;
4
5 struct Process{ int pid, at, bt, priority, ct, tat, wt, queue_no, rem_bt; bool finished = false, vis
    = false;
6 void show(){ cout << pid << "\t" << at << "\t" << bt << "\t" << priority << "\t\t\t" << ct << "\t" <<
    tat << "\t" << wt << endl; }
7 };
8 struct Gantt{ int pid, start, end; };
9
10 void showGanttChart(vector<Gantt> g){
11     cout << "\n\nGantt Chart\n\n";
12     for(auto x : g){ if(x.pid == -1) cout << "|Idle"; else cout << "| P" << x.pid << " "; }
13     cout << "\n" << g[0].start;
14     for(auto x : g) cout << setw(5) << x.end;
15     nl;
16 }
17
18 int32_t main(){
19     int n = 0; cin >> n;
20     vector<Process> p(n); vector<Gantt> g;
21     for(int i = 0; i < n; i++){
22         p[i].pid = i + 1; cin >> p[i].at >> p[i].bt >> p[i].queue_no;
23         p[i].rem_bt = p[i].bt; p[i].priority = p[i].queue_no;
24     }
25
26     queue<int> q1, q2; int tq = 2, curTime = 0, completed = 0;
27     auto doneTask = [&](int i){
28         p[i].ct = curTime; p[i].tat = p[i].ct - p[i].at; p[i].wt = p[i].tat - p[i].bt; p[i].finished =
            true; completed++;
29     };
30
31     while(completed < n){
32         for(int i = 0; i < n; i++){
33             if(!p[i].finished and p[i].at <= curTime and !p[i].vis){
34                 if(p[i].queue_no == 1) q1.push(i); else q2.push(i);
35                 p[i].vis = true;
36             }
37         }
38         if(!q1.empty()){
39             int i = q1.front(); q1.pop();
40             g.push_back({p[i].pid, curTime, curTime + p[i].bt});
41             curTime += p[i].bt; doneTask(i);
42         }else if(!q2.empty()){
43             int i = q2.front(); q2.pop();
44             int runTime = min(tq, p[i].rem_bt);
45             g.push_back({p[i].pid, curTime, curTime + runTime});
46             curTime += runTime; p[i].rem_bt -= runTime;
47             if(p[i].rem_bt == 0) doneTask(i); else q2.push(i);
48         }else{
49             g.push_back({-1, curTime, curTime + 1}); curTime++;
50         }
51     }
52
53     cout << "\nPID\tAT\tBT\tQueue\t\tCT\tTAT\tWT\n";
54     double totTat = 0, totWt = 0;
55     for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
56     cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
57     showGanttChart(g);
58 }
```

8 Multilevel Feedback Queue (MLFQ)

A dynamic queue system. Queue 1: RR (tq=2) → Queue 2: RR (tq=4) → Queue 3: SJF. Processes demote if they exhaust their time quantum.

Input & Output Format

Input: Integer n. Next n lines: AT & BT.

Output: Table of PID, AT, BT, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define nl cout << endl;
4
5 struct Process{ int pid, at, bt, priority, ct, tat, wt, queue_no, rem_bt; bool finished = false, vis
   = false;
6 void show(){ cout << pid << "\t" << at << "\t" << bt << "\t" << ct << "\t" << tat << "\t" << wt << endl
   ; }
7 };
8 struct Gantt{ int pid, start, end; };
9
10 void showGanttChart(vector<Gantt> g){
11     cout << "\n\nGantt Chart\n\n";
12     for(auto x : g){ if(x.pid == -1) cout << "| Idle"; else cout << "| P" << x.pid << " "; }
13     cout << "\n" << g[0].start;
14     for(auto x : g) cout << setw(5) << x.end;
15     nl;
16 }
17
18 int32_t main(){
19     int n = 0; cin >> n;
20     vector<Process> p(n); vector<Gantt> g;
21     for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt; p[i].rem_bt = p[i].bt; }
22
23     queue<int> q1, q2, q3; int tq1 = 2, tq2 = 4, curTime = 0, completed = 0;
24
25     auto doneTask = [&](int i){
26         p[i].ct = curTime; p[i].tat = p[i].ct - p[i].at; p[i].wt = p[i].tat - p[i].bt; p[i].finished =
         true; completed++;
27     };
28
29     while(completed < n){
30         for(int i = 0; i < n; i++){
31             if(!p[i].finished and p[i].at <= curTime and !p[i].vis){ q1.push(i); p[i].vis = true; }
32         }
33
34         if(!q1.empty()){
35             int i = q1.front(); q1.pop();
36             int runTime = min(tq1, p[i].rem_bt);
37             g.push_back({p[i].pid, curTime, curTime + runTime});
38             curTime += runTime; p[i].rem_bt -= runTime;
39             if(p[i].rem_bt == 0) doneTask(i); else q2.push(i);
40         }else if(!q2.empty()){
41             int i = q2.front(); q2.pop();
42             int runTime = min(tq2, p[i].rem_bt);
43             g.push_back({p[i].pid, curTime, curTime + runTime});
44             curTime += runTime; p[i].rem_bt -= runTime;
45             if(p[i].rem_bt == 0) doneTask(i); else q3.push(i);
46         }else if(!q3.empty()){
47             int i = q3.front(); q3.pop();
48             g.push_back({p[i].pid, curTime, curTime + p[i].rem_bt});
49             curTime += p[i].rem_bt; p[i].rem_bt = 0; doneTask(i);
50         }else {
51             g.push_back({-1, curTime, curTime + 1}); curTime++;
52         }
53     }
54
55     cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
56     double totTat = 0, totWt = 0;
57     for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
58     cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
59     showGanttChart(g);
60 }
```